

C++ DSA Mini Project – Problem Statements

13 Group-wise Projects

These projects are designed around the concepts covered in class: STL, 1D Arrays, 2D Arrays, Strings, Stack, Queue and Structures. Each group has a different problem statement.

Common Project Requirements

- Develop the project in C++.
- Create a menu-driven console application.
- Use functions to divide the program into logical operations.
- Use structures wherever multiple attributes belong to one entity.
- Use at least one STL container where appropriate.
- Include searching and sorting wherever relevant to the problem.
- Validate user input and handle invalid operations gracefully.
- Do not use a single main function containing the complete logic.
- Code should be readable, properly indented and meaningfully commented.
- Each group should be able to explain the DSA concepts used in its project during the demo.

Suggested Evaluation

- Problem understanding and features – 20 marks
- Correct use of DSA concepts – 20 marks
- Code quality, functions and modularity – 15 marks
- Input validation and edge cases – 10 marks
- Output/UI and usability – 10 marks
- Demo and explanation – 15 marks
- Team contribution and documentation – 10 marks

Group 1 – Bank Management System

Build a menu-driven C++ application to manage bank customers and their accounts.

Problem Statement

Your team is required to design and implement a C++ application for bank management system. The application should be menu-driven and should allow the user to perform the required operations through clear options. The implementation must demonstrate appropriate use of the DSA concepts taught in class.

Mandatory Features

1. Add a new customer/account
2. Display all customers
3. Search customer by account number
4. Deposit money
5. Withdraw money with sufficient-balance validation
6. Display account balance
7. Find the customer with the highest balance
8. Sort customers by balance

DSA Concepts Expected

- Structures
- Arrays / STL containers
- Strings
- Searching
- Sorting

Implementation Expectations

Use separate functions for major operations. Handle cases such as empty records, invalid IDs, duplicate entries, unavailable resources, and invalid input wherever applicable. Students should be prepared to explain why each selected data structure was used.

Suggested Menu

- 1. Add / Create record
- 2. Display records
- 3. Search
- 4. Update / Process
- 5. Delete / Cancel where applicable
- 6. Sort / Report where applicable
- 7. Exit

Demo Requirement

During the demonstration, the group should explain the problem, show the major features, demonstrate at least two edge cases, and explain the data structures and algorithms used.

Group 2 – Student Result Management System

Build a system to maintain student academic records and generate results.

Problem Statement

Your team is required to design and implement a C++ application for student result management system. The application should be menu-driven and should allow the user to perform the required operations through clear options. The implementation must demonstrate appropriate use of the DSA concepts taught in class.

Mandatory Features

9. Add student details including roll number, name and subject marks
10. Calculate total and percentage
11. Assign grades
12. Display all students
13. Search student by roll number
14. Display topper
15. Sort students by percentage
16. Display failed students

DSA Concepts Expected

- Structures
- Arrays / STL containers
- Strings
- Searching
- Sorting

Implementation Expectations

Use separate functions for major operations. Handle cases such as empty records, invalid IDs, duplicate entries, unavailable resources, and invalid input wherever applicable. Students should be prepared to explain why each selected data structure was used.

Suggested Menu

- 1. Add / Create record
- 2. Display records
- 3. Search
- 4. Update / Process
- 5. Delete / Cancel where applicable
- 6. Sort / Report where applicable
- 7. Exit

Demo Requirement

During the demonstration, the group should explain the problem, show the major features, demonstrate at least two edge cases, and explain the data structures and algorithms used.

Group 3 – Hospital Patient Management System

Build a patient management system where patients are registered and handled according to their waiting order.

Problem Statement

Your team is required to design and implement a C++ application for hospital patient management system. The application should be menu-driven and should allow the user to perform the required operations through clear options. The implementation must demonstrate appropriate use of the DSA concepts taught in class.

Mandatory Features

17. Register a patient
18. Add patient to the waiting queue
19. Call/process the next patient
20. Display all waiting patients
21. Search patient by ID
22. Display patient details
23. Count waiting patients
24. Handle emergency patients separately

DSA Concepts Expected

- Queue
- Structures
- Arrays / STL containers
- Strings

Implementation Expectations

Use separate functions for major operations. Handle cases such as empty records, invalid IDs, duplicate entries, unavailable resources, and invalid input wherever applicable. Students should be prepared to explain why each selected data structure was used.

Suggested Menu

- 1. Add / Create record
- 2. Display records
- 3. Search
- 4. Update / Process
- 5. Delete / Cancel where applicable
- 6. Sort / Report where applicable
- 7. Exit

Demo Requirement

During the demonstration, the group should explain the problem, show the major features, demonstrate at least two edge cases, and explain the data structures and algorithms used.

Group 4 – Railway Ticket Reservation System

Create a simplified railway reservation system for searching trains, booking/cancelling tickets and managing waiting passengers.

Problem Statement

Your team is required to design and implement a C++ application for railway ticket reservation system. The application should be menu-driven and should allow the user to perform the required operations through clear options. The implementation must demonstrate appropriate use of the DSA concepts taught in class.

Mandatory Features

25. Add/display train information
26. Search train by number or destination
27. Book a ticket
28. Cancel a ticket
29. Display available seats
30. Display passenger details
31. Maintain a waiting list using a queue

DSA Concepts Expected

- Structures
- Arrays
- Queue
- Strings
- Searching

Implementation Expectations

Use separate functions for major operations. Handle cases such as empty records, invalid IDs, duplicate entries, unavailable resources, and invalid input wherever applicable. Students should be prepared to explain why each selected data structure was used.

Suggested Menu

- 1. Add / Create record
- 2. Display records
- 3. Search
- 4. Update / Process
- 5. Delete / Cancel where applicable
- 6. Sort / Report where applicable
- 7. Exit

Demo Requirement

During the demonstration, the group should explain the problem, show the major features, demonstrate at least two edge cases, and explain the data structures and algorithms used.

Group 5 – Library Management System

Create a library application to manage books and student/member borrowing records.

Problem Statement

Your team is required to design and implement a C++ application for library management system. The application should be menu-driven and should allow the user to perform the required operations through clear options. The implementation must demonstrate appropriate use of the DSA concepts taught in class.

Mandatory Features

32. Add a book
33. Display all books
34. Search book by title, ID or author
35. Issue a book
36. Return a book
37. Display available/issued books
38. Search books by author
39. Sort books by title

DSA Concepts Expected

- Structures
- Arrays / STL containers
- Strings
- Searching
- Sorting

Implementation Expectations

Use separate functions for major operations. Handle cases such as empty records, invalid IDs, duplicate entries, unavailable resources, and invalid input wherever applicable. Students should be prepared to explain why each selected data structure was used.

Suggested Menu

- 1. Add / Create record
- 2. Display records
- 3. Search
- 4. Update / Process
- 5. Delete / Cancel where applicable
- 6. Sort / Report where applicable
- 7. Exit

Demo Requirement

During the demonstration, the group should explain the problem, show the major features, demonstrate at least two edge cases, and explain the data structures and algorithms used.

Group 6 – Online Shopping Cart System

Build a console-based shopping application where users can browse products and manage a shopping cart.

Problem Statement

Your team is required to design and implement a C++ application for online shopping cart system. The application should be menu-driven and should allow the user to perform the required operations through clear options. The implementation must demonstrate appropriate use of the DSA concepts taught in class.

Mandatory Features

40. Display products
41. Search product
42. Add product to cart
43. Remove product from cart
44. Update quantity
45. Calculate subtotal and final bill
46. Apply a discount based on a defined rule
47. Display the final invoice

DSA Concepts Expected

- Structures
- Arrays / STL containers
- Strings
- Searching

Implementation Expectations

Use separate functions for major operations. Handle cases such as empty records, invalid IDs, duplicate entries, unavailable resources, and invalid input wherever applicable. Students should be prepared to explain why each selected data structure was used.

Suggested Menu

- 1. Add / Create record
- 2. Display records
- 3. Search
- 4. Update / Process
- 5. Delete / Cancel where applicable
- 6. Sort / Report where applicable
- 7. Exit

Demo Requirement

During the demonstration, the group should explain the problem, show the major features, demonstrate at least two edge cases, and explain the data structures and algorithms used.

Group 7 – Movie Ticket Booking System

Create a movie ticket booking application using a 2D array to represent cinema seats.

Problem Statement

Your team is required to design and implement a C++ application for movie ticket booking system. The application should be menu-driven and should allow the user to perform the required operations through clear options. The implementation must demonstrate appropriate use of the DSA concepts taught in class.

Mandatory Features

48. Display movie and seat layout
49. Book a selected seat
50. Prevent booking an already occupied seat
51. Cancel a booking
52. Display booked and available seats
53. Calculate ticket price
54. Search booking using customer name

DSA Concepts Expected

- 2D Arrays – mandatory for seats
- Structures
- Strings
- Arrays
- Searching

Implementation Expectations

Use separate functions for major operations. Handle cases such as empty records, invalid IDs, duplicate entries, unavailable resources, and invalid input wherever applicable. Students should be prepared to explain why each selected data structure was used.

Suggested Menu

- 1. Add / Create record
- 2. Display records
- 3. Search
- 4. Update / Process
- 5. Delete / Cancel where applicable
- 6. Sort / Report where applicable
- 7. Exit

Demo Requirement

During the demonstration, the group should explain the problem, show the major features, demonstrate at least two edge cases, and explain the data structures and algorithms used.